

Model Impedance

**Object–Relational Impedance
Mismatch**

Impedance Mismatch in DBMS

- In a database management system (DBMS), impedance mismatch refers to the discrepancy between the data structures used in the **application code (object model)** and those used in the database (relational model)

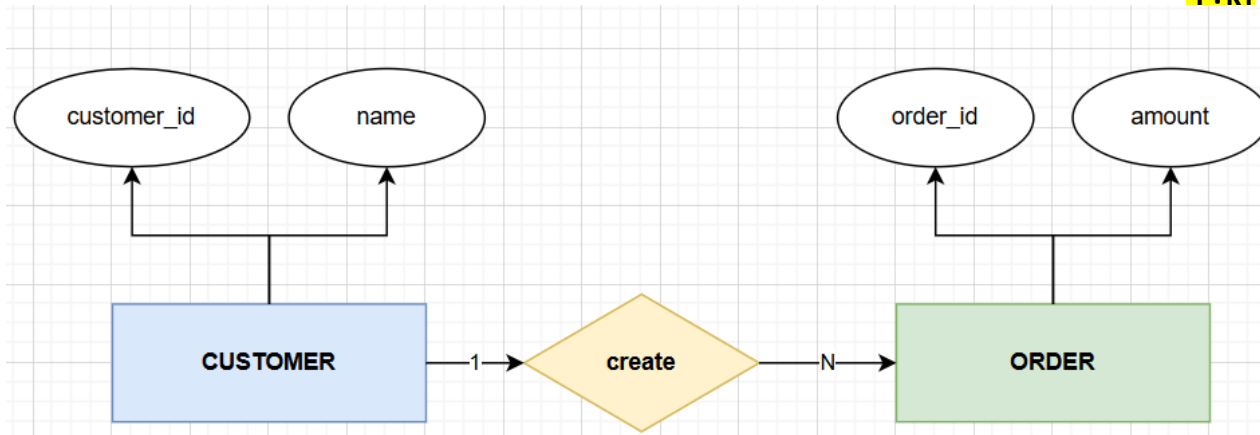
<https://www.geeksforgeeks.org/dbms/impedance-mismatch-in-dbms/>

Relational model

Concept

- relation
- tuple
- attribute
- primary key
- foreign key
- constraints

1..N



CUSTOMER

customerId	name
1	Somchai
2	Somsak

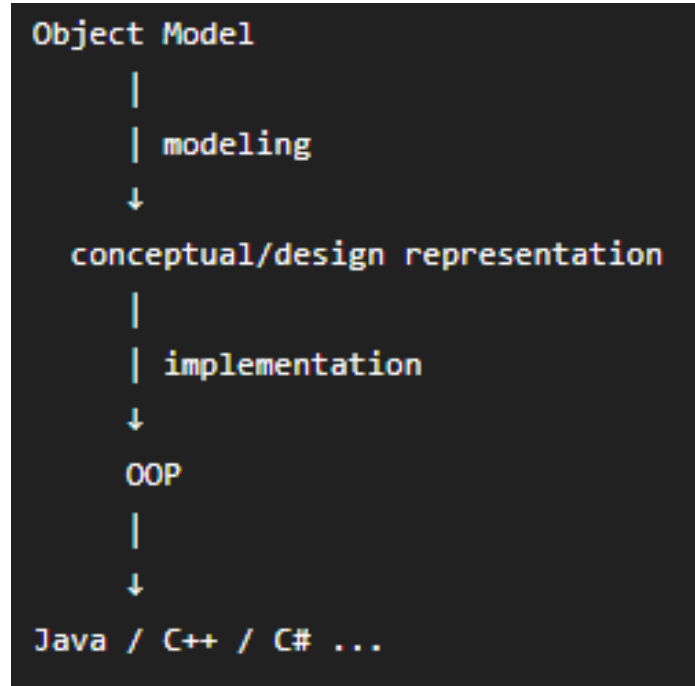
ORDER

orderId	customerId	amount
101	1	500.00
102	1	700.00
103	2	300.00

Object Model

Object model concepts represents in form **objects/classes**

- object identity
- reference
- encapsulation
- inheritance
- polymorphism
- object graph



Model Impedance

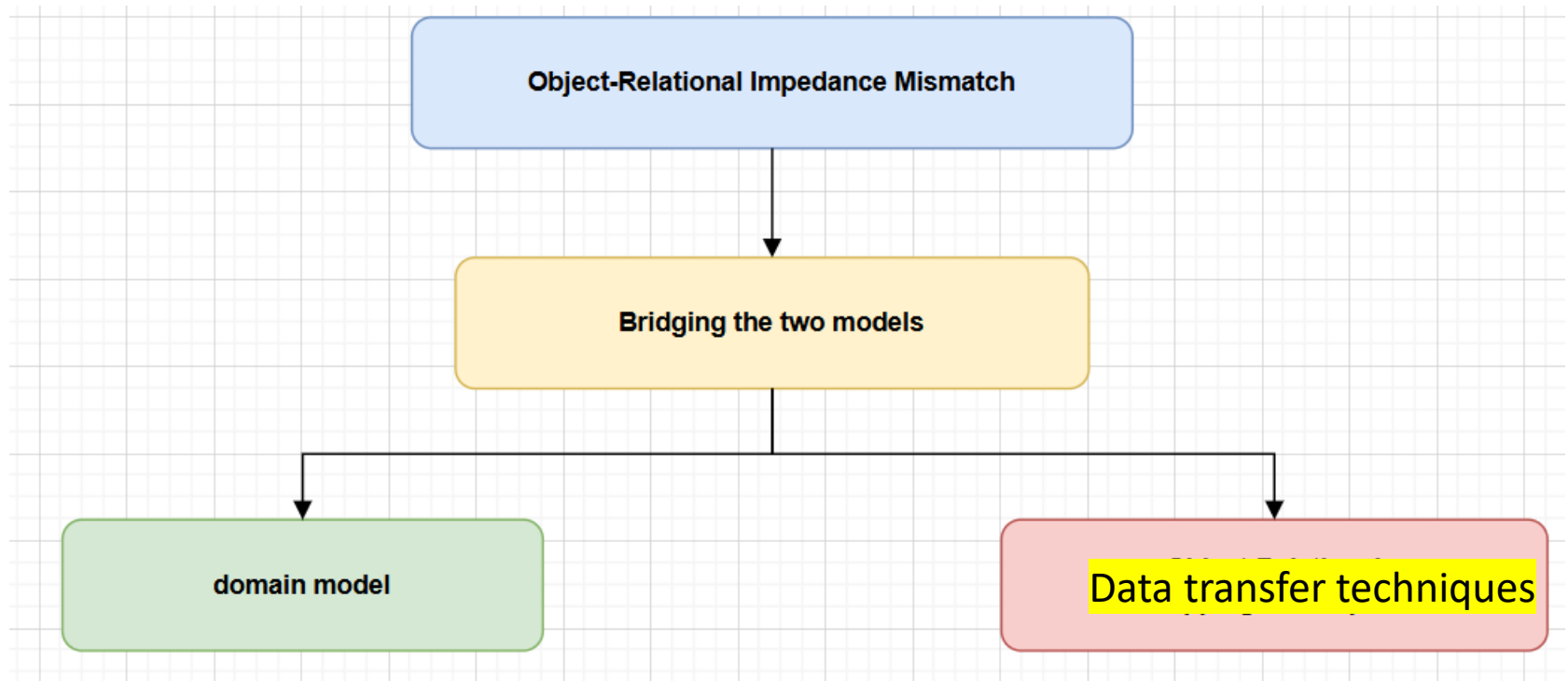
Model impedance is the incompatibility between the concepts and structures of the Object Model and the Relational Model, which makes it impossible to represent one model directly in terms of the other.

issue	Object Model	Relational Model
Data instance	Object	Row/Tuple
Structure	Graph of objects	Set of flat tables
Identity	Object reference / pointer	Primary key (value-based)
Inheritance	Class hierarchy	No has native support

ORM

Object-relational mapping (ORM) is a technique for bridging the object-oriented model and the relational model. It involves:

- 1.Designing object-oriented classes** that represent the application or domain data.
- 2.Defining mappings** between those classes and relational database structures, such as tables, columns, primary keys, foreign keys, and relationships.
- 3.Providing mechanisms for accessing and persisting objects** in the relational database.



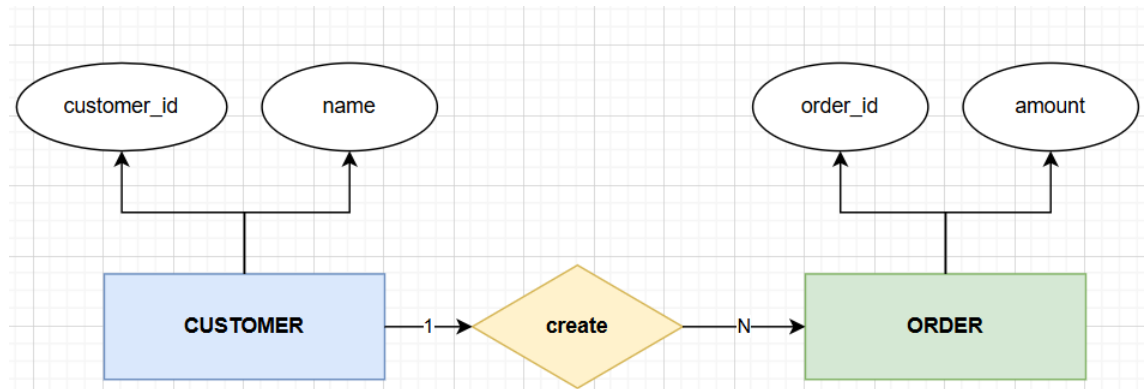
การสร้าง Domain model (Defining mappings)

1:N

- reference ID

```
class Customer {  
    long customerId;  
    String name;  
}
```

```
class Order {  
    long orderId;  
    long customerId; // reference ID  
    double amount;  
}
```



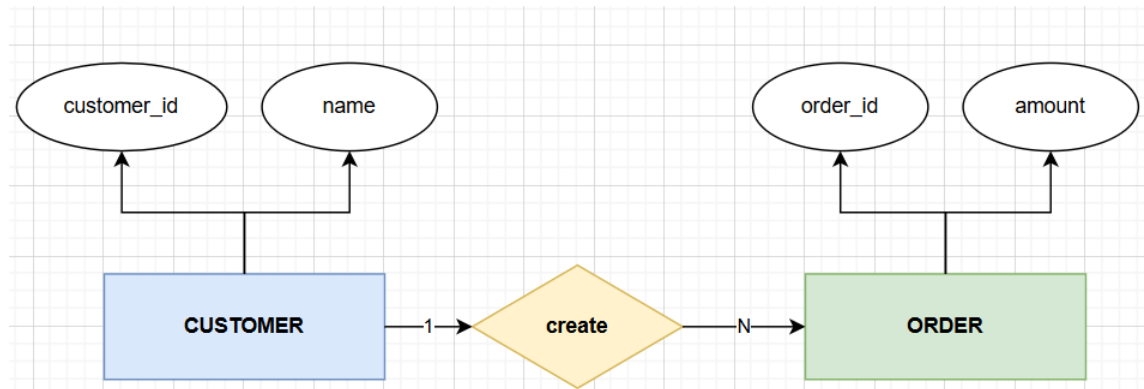
การสร้าง Domain model

1:N

- Object Reference

```
class Customer {  
    long customerId;  
    String name;  
}
```

```
class Order {  
    long orderId;  
    Customer customer;  
    double amount;  
}
```



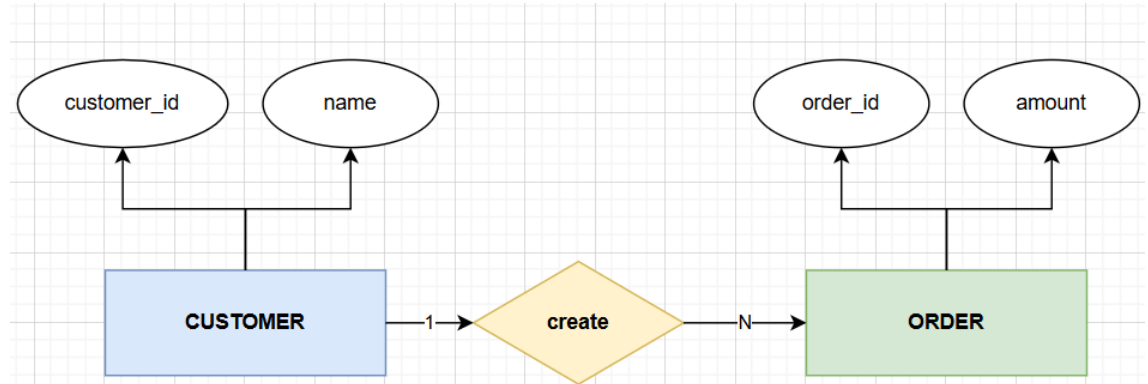
การสร้าง Domain model

1:N

- Collection / Object Navigation

```
class Customer {  
    long customerId;  
    String name;  
  
    List<Order> orders;  
}
```

```
class Order {  
    long orderId;  
    double amount;  
}
```



การสร้าง Domain model

1:N

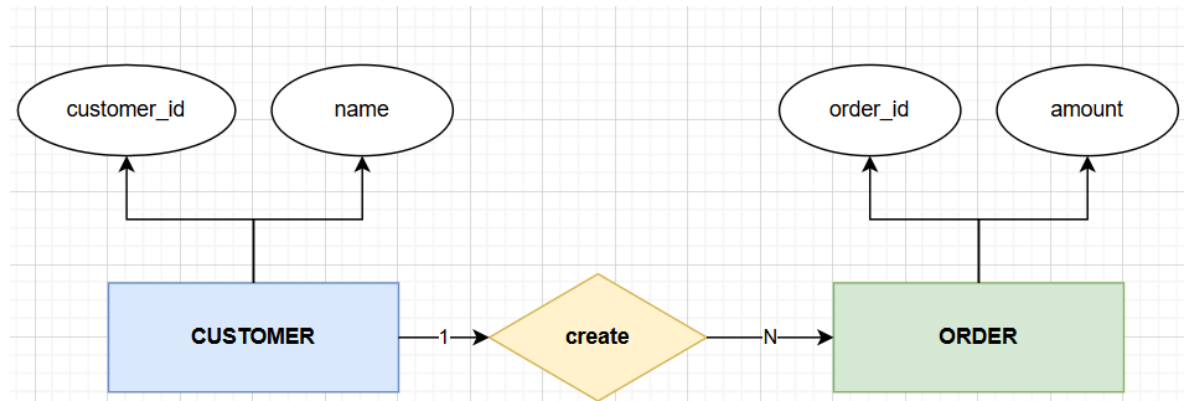
- Bidirectional Association

```
class Customer {  
    long customerId;  
    String name;
```

```
    List<Order> orders;
```

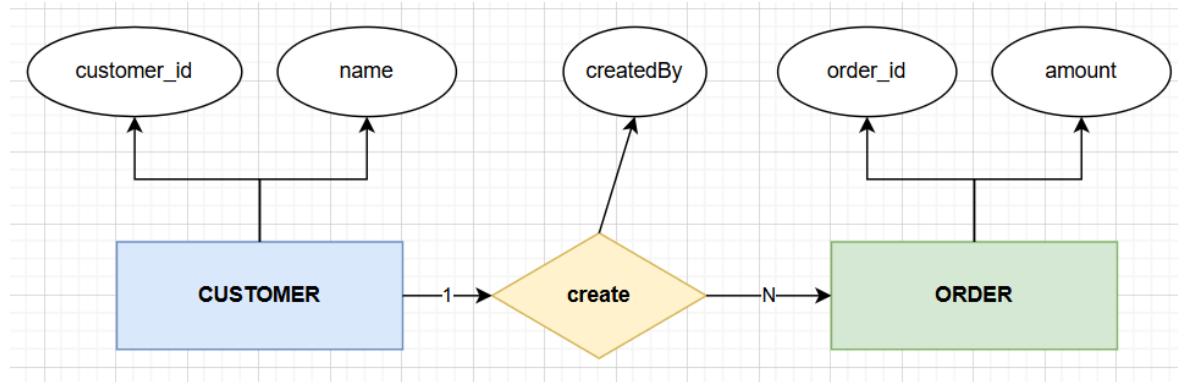
```
}
```

```
class Order {  
    long orderId;  
    Customer customer;  
    double amount;  
}
```



1:N กรณีมี attribute on relation

```
class Customer {  
    long customerId;  
    String name;  
}
```



```
class Order {  
    long orderId;  
    long customerId; // reference ID  
    double amount;  
    string createdBy;  
}
```

การสร้าง Domain model

กรณี #1 : relation has no attribute

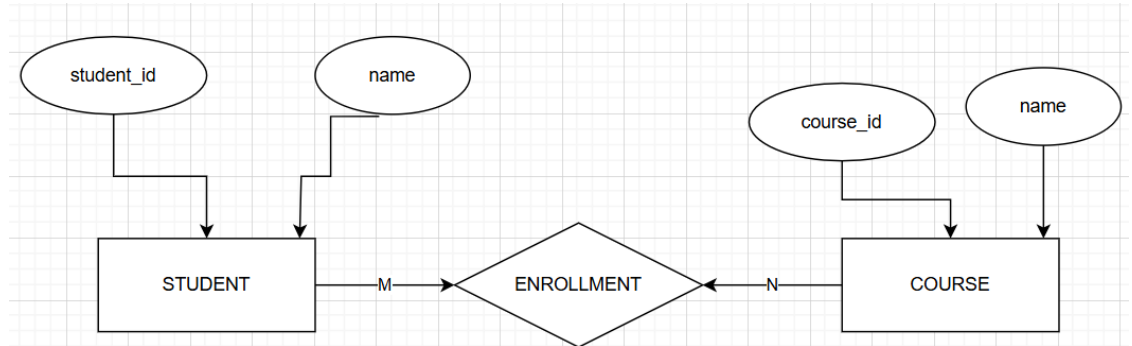
N:M

STUDENT

student_id	name
101	Somchai
102	Suda
103	Anan

COURSE

course_id	name
501	Database
502	Programming
503	Software Engineering



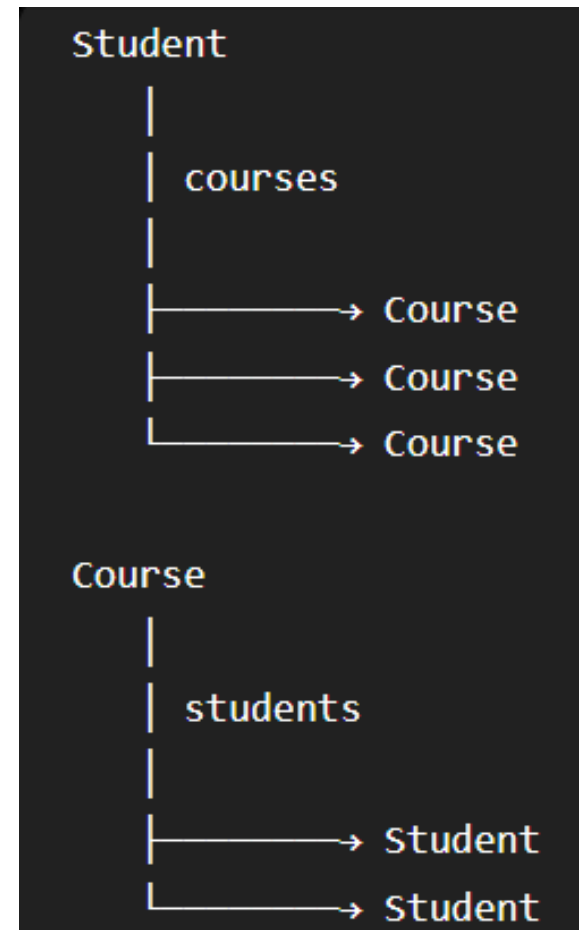
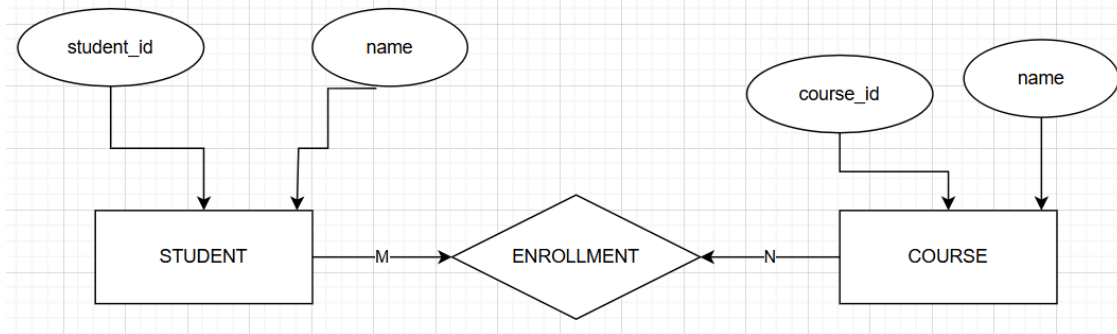
ENROLLMENT

student_id	course_id
101	501
101	502
102	501
102	503
103	502
103	503

กรณี #1 : relation has no attribute

Collection

```
class Student {  
    long studentId;  
    String name;  
    List<Course> courses;  
}  
  
class Course {  
    long courseId;  
    String name;  
    List<Student> students;  
}
```

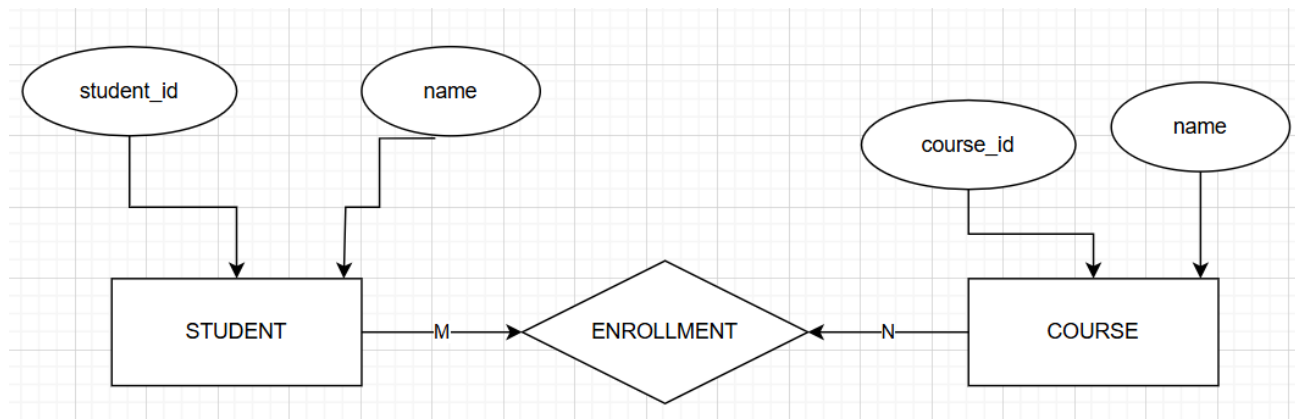


กรณี #1 : relation has no attribute

```
class Student {  
    long studentId;  
    String name;  
    List<Enrollment> enrollments;  
}
```

```
class Course {  
    long courseId;  
    String name;  
    List<Enrollment> enrollments;  
}
```

```
class Enrollment {  
    Student student;  
    Course course;  
}
```



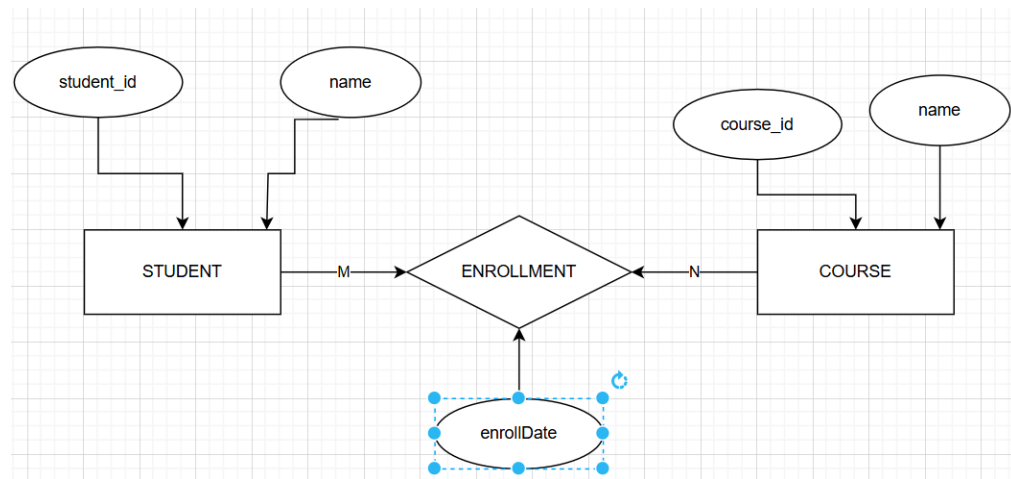
กรณี #1 : relation has some attributes

STUDENT

student_id	name
101	Somchai
102	Suda
103	Anan

COURSE

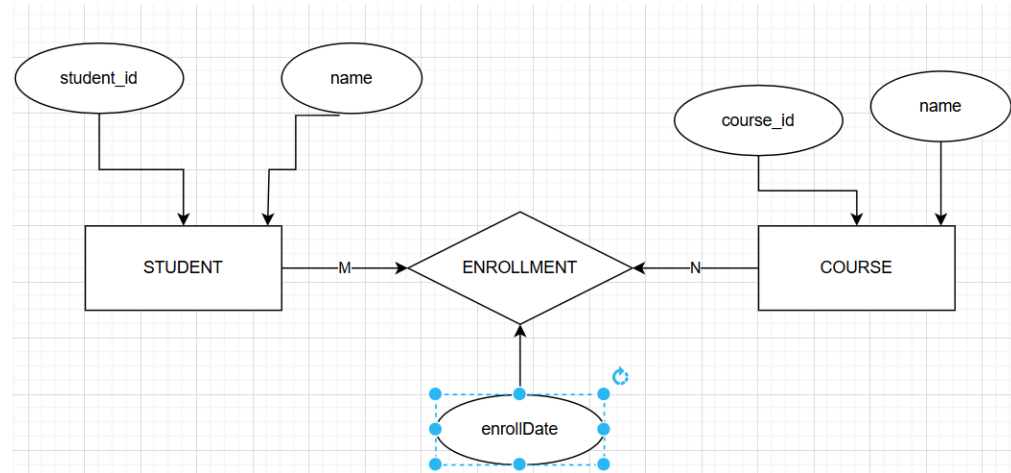
course_id	name
501	Database
502	Programming
503	Software Engineering



ENROLLMENT

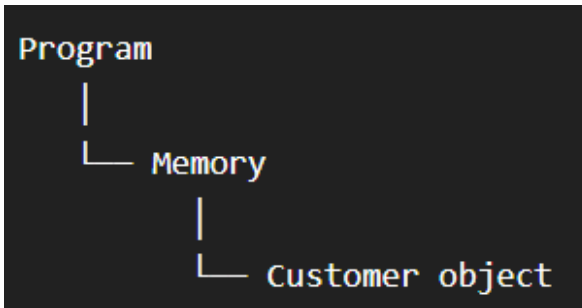
student_id	course_id	enroll_date
101	501	2026-09-01
101	502	2026-09-02
102	501	2026-09-03

```
class Student {  
    long studentId;  
    String name;  
    List<Enrollment> enrollments;  
}  
class Course {  
    long courseId;  
    String name;  
    List<Enrollment> enrollments;  
}  
class Enrollment {  
    Student student;  
    Course course;  
    Date enrollDate;  
}
```

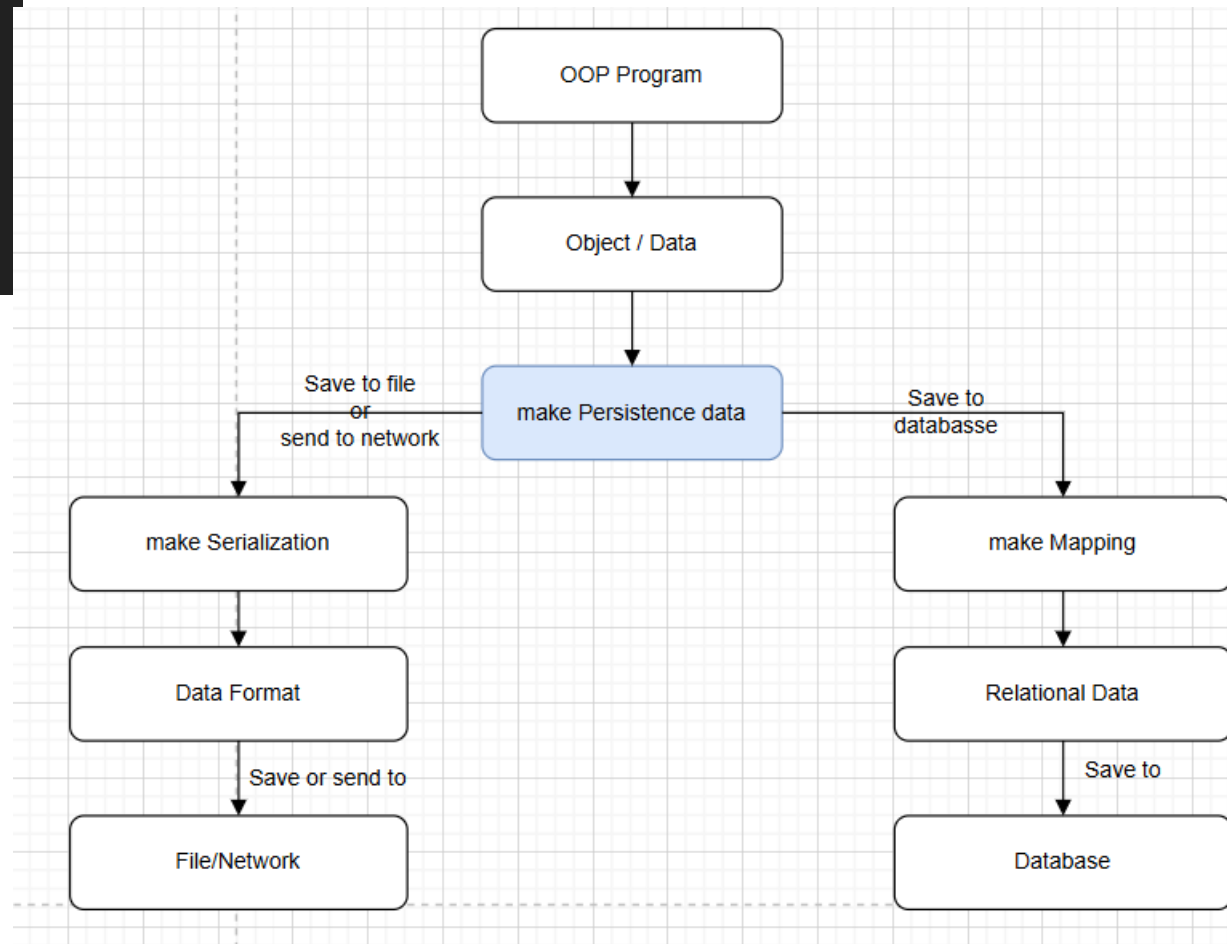


Data Persistence

Customer c = new Customer(1001, "John");



เมื่อ program จบ object
นี้ก็หายไปจาก memory

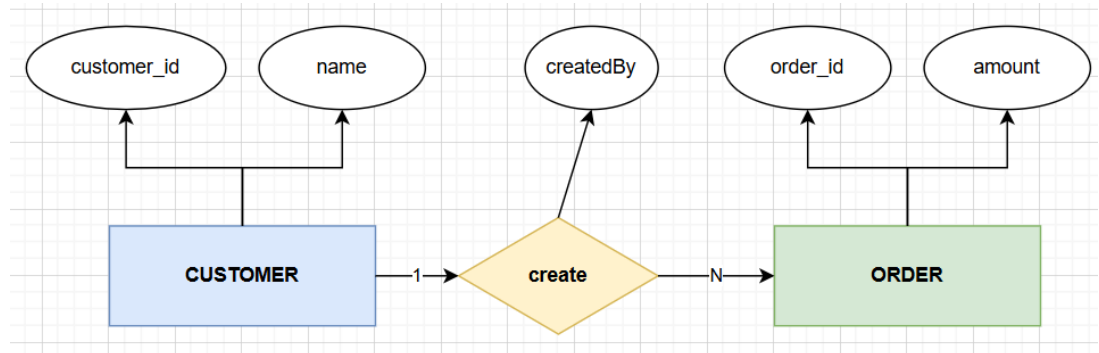


Entity Class

An entity class represents objects that have persistent state and an object identifier.

```
class Customer {  
    long customerId;  
    String name;  
}
```

```
class Order {  
    long orderId;  
    long customerId; // reference ID  
    double amount;  
}
```



Data transfer techniques

(Providing mechanisms for accessing and persisting objects)

Data Gateway —

- A **Data Source Architectural Pattern** used to encapsulate access to a data source.
- An object that encapsulates access to a data source and provides data access operations.
- In an object-relational application, it can be used as part of the mechanism for accessing relational data represented by objects.

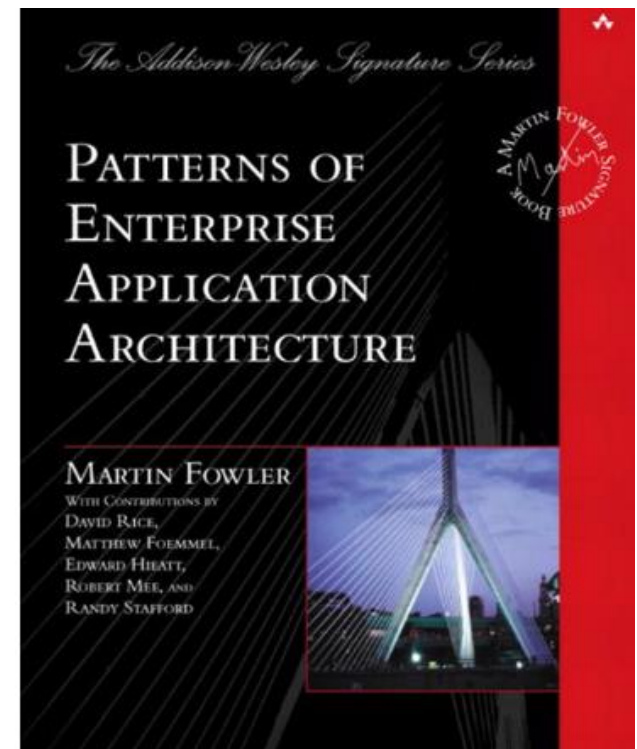
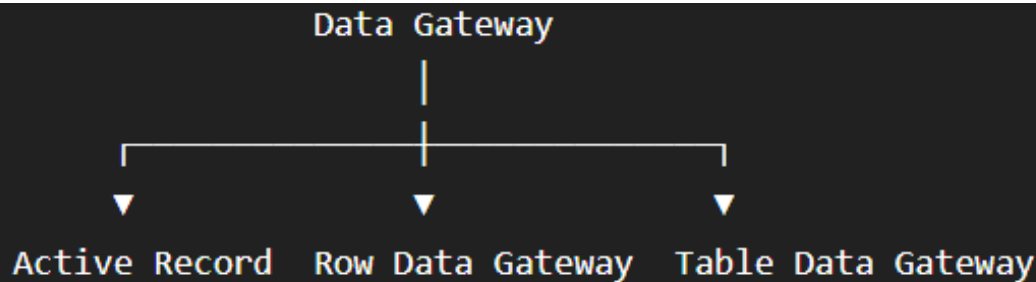
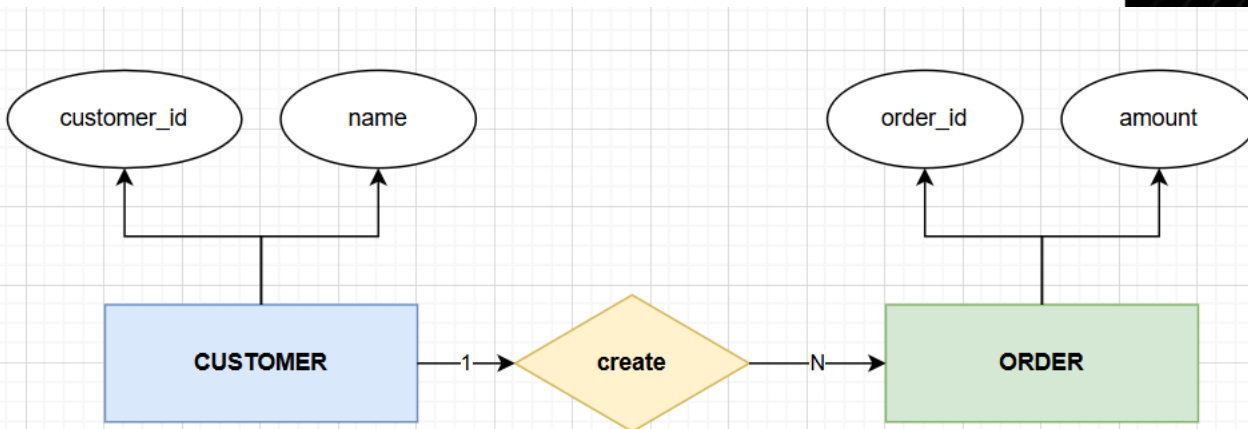
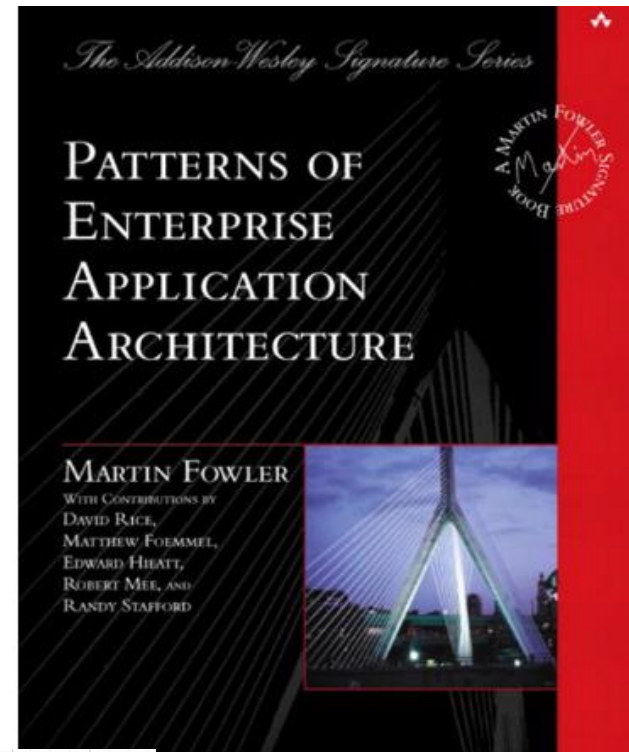
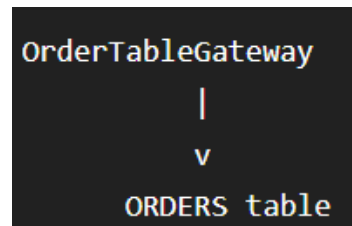
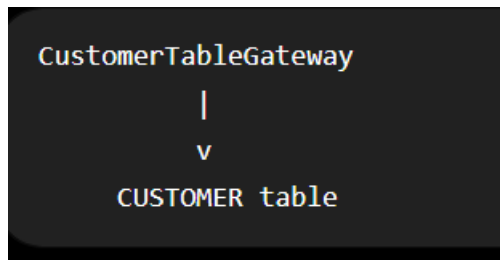


Table Data Gateway

A Table Data Gateway holds all the SQL for accessing a single table or view:
selects, inserts, updates, and deletes.



```
class Customer {  
    private long customerId;  
    private String name;  
    private List<Order> orders;  
  
    public long getCustomerId() {  
        return customerId;  
    }  
    public void setCustomerId(long customerId) {  
        this.customerId = customerId;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public List<Order> getOrders() {  
        return orders;  
    }  
    public void setOrders(List<Order> orders) {  
        this.orders = orders;  
    }  
}
```

```
class Order {  
    private long orderId;  
    private double amount;  
  
    public long getOrderId() {  
        return orderId;  
    }  
    public void setOrderId(long orderId) {  
        this.orderId = orderId;  
    }  
    public double getAmount() {  
        return amount;  
    }  
    public void setAmount(double amount) {  
        this.amount = amount;  
    }  
}
```

```
import java.sql.*;
import java.util.*;
```

```
class CustomerTableGateway {
    private Connection connection;
    public CustomerTableGateway(Connection connection) {
        this.connection = connection;
    }
    // SELECT
    public Customer findById(long id) throws SQLException {

        String sql = ""
            SELECT customer_id, name, email
            FROM CUSTOMER
            WHERE customer_id = ?
        """;

        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setLong(1, id);
            try (ResultSet rs = ps.executeQuery()) {

                if (rs.next()) {
                    return new Customer(
                        rs.getLong("customer_id"),
                        rs.getString("name"),
                        rs.getString("email")
                    );
                }
            }
        }

        return null;
    }
}
```

```
// INSERT
    public void insert(Customer customer)
        throws SQLException {

        String sql = ""
            INSERT INTO CUSTOMER
            (customer_id, name, email)
            VALUES (?, ?, ?)
        """;

        try (PreparedStatement ps =
            connection.prepareStatement(sql)) {

            ps.setLong(1, customer.getCustomerId());
            ps.setString(2, customer.getName());
            ps.setString(3, customer.getEmail());

            ps.executeUpdate();
        }
    }
}
```

```
class OrderTableGateway {
    private Connection connection;
    public OrderTableGateway(Connection connection) {
        this.connection = connection;
    }
    public List<Order> findByCustomer( long customerId) throws SQLException {
        String sql =
            "SELECT order_id, amount " +
            "FROM orders " +
            "WHERE customer_id = ?";
        PreparedStatement ps =
            connection.prepareStatement(sql);
        ps.setLong(1, customerId);
        ResultSet rs = ps.executeQuery();

        List<Order> orders = new ArrayList<>();
        while (rs.next()) {
            Order order = new Order();
            order.setOrderId(
                rs.getLong("order_id"));
            order.setAmount(
                rs.getDouble("amount"));
            orders.add(order);
        }

        return orders;
    }
}
```

การเรียกใช้งาน

```
CustomerTableGateway customerGateway = new  
CustomerTableGateway(connection);
```

```
OrderTableGateway orderGateway = new OrderTableGateway(connection);
```

```
Customer customer = customerGateway.findById(1);
```

```
List<Order> orders = orderGateway.findByCustomer( customer.getId());  
customer.setOrders(orders);
```

//business logic ควรต้องอยู่อีก layer หรือ class อื่น

```
class CustomerService {
```

```
    public boolean isVip(CustomerTableGateway customer) {
```

```
        return ...;
```

```
    }
```

```
}
```

Row Data Gateway

An object that acts as a gateway to a single record in a data source. There is one instance per row (Each Row Data Gateway instance stores the data of one database row.)

Table Data Gateway



table



หลาย rows

Row Data Gateway



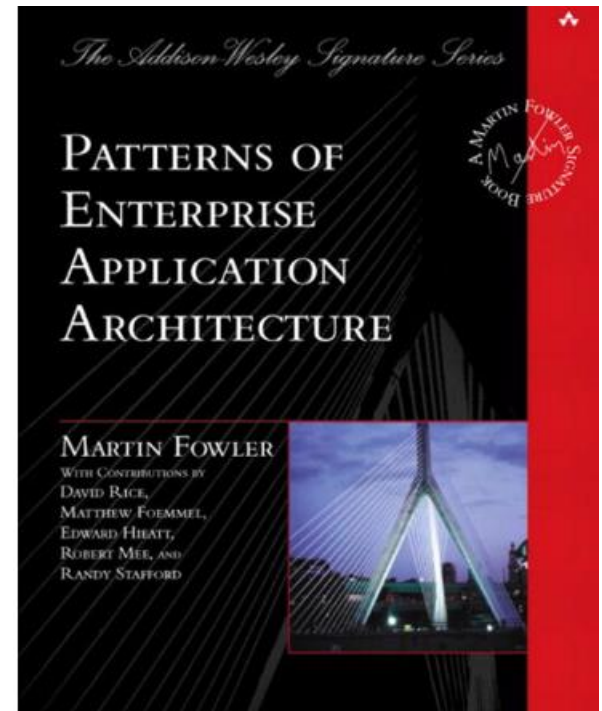
row เดียว

CustomerRowGateway

```
|  
|— customerId  
|— name  
|— insert()  
|— update()  
|— delete()
```

OrderRowGateway

```
|  
|— orderId  
|— customerId  
|— amount  
|— insert()  
|— update()  
|— delete()
```



```
class CustomerRowGateway {
```

```
    private long customerId;
```

```
    private String name;
```

```
    public CustomerRowGateway(long customerId, String name) {
```

```
        this.customerId = customerId;
```

```
        this.name = name;
```

```
    }
```

```
    public static CustomerRowGateway find(Connection connection, long id)
```

```
        throws SQLException {
```

```
        String sql = "SELECT customer_id, name FROM CUSTOMER WHERE customer_id = ?";
```

```
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
```

```
            ps.setLong(1, id);
```

```
            try (ResultSet rs = ps.executeQuery()) {
```

```
                if (rs.next()) {
```

```
                    return new CustomerRowGateway(
```

```
                        rs.getLong("customer_id"),
```

```
                        rs.getString("name")
```

```
                    );
```

```
                }
```

```
            }
```

```
        }
```

```
        return null;
```

```
    }
```

CustomerRowGateway

- |
- ├─ customerId
- ├─ name
- ├─ insert()
- ├─ update()
- └─ delete()

OrderRowGateway

- |
- ├─ orderId
- ├─ customerId
- ├─ amount
- ├─ insert()
- ├─ update()
- └─ delete()

```
class CustomerRowGateway {
```

```
    private long customerId;
```

```
    private String name;
```

```
    :
```

```
    public void update(Connection connection)  
        throws SQLException {
```

```
        String sql = "UPDATE CUSTOMER SET name = ? WHERE customer_id = ? ";
```

```
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
```

```
            ps.setString(1, name);
```

```
            ps.setLong(2, customerId);
```

```
            ps.executeUpdate();
```

```
        }
```

```
    }
```

```
}
```

CustomerRowGateway

|

├─ customerId

├─ name

├─ insert()

├─ update()

└─ delete()

OrderRowGateway

|

├─ orderId

├─ customerId

├─ amount

├─ insert()

├─ update()

└─ delete()

```

class OrderRowGateway {
    private long orderId;
    private long customerId;
    private double amount;
    public OrderRowGateway( long orderId, long customerId, double amount) {
        this.orderId = orderId;
        this.customerId = customerId;
        this.amount = amount;
    }

```

```

    public static List<OrderRowGateway> findByCustomer( Connection connection, long customerId)
        throws SQLException {
        String sql = "SELECT order_id, customer_id, amount FROM orders WHERE customer_id = ?";
        PreparedStatement ps = connection.prepareStatement(sql);
        ps.setLong(1, customerId);
        ResultSet rs = ps.executeQuery();

        List<OrderRowGateway> result = new ArrayList<>();
        while (rs.next()) {
            result.add(
                new OrderRowGateway(
                    rs.getLong("order_id"),
                    rs.getLong("customer_id"),
                    rs.getDouble("amount")
                )
            );
        }
    }

```

CustomerRowGateway

```

|
|— customerId
|— name
|— insert()
|— update()
|— delete()

```

OrderRowGateway

```

|
|— orderId
|— customerId
|— amount
|— insert()
|— update()

```

การเรียกใช้งาน :

```
CustomerRowGateway customer = CustomerRowGateway.find(  
    connection,1);
```

```
List<OrderRowGateway> orders = OrderRowGateway.findByCustomer(  
    connection,  
    customer.getCustomerId() );
```

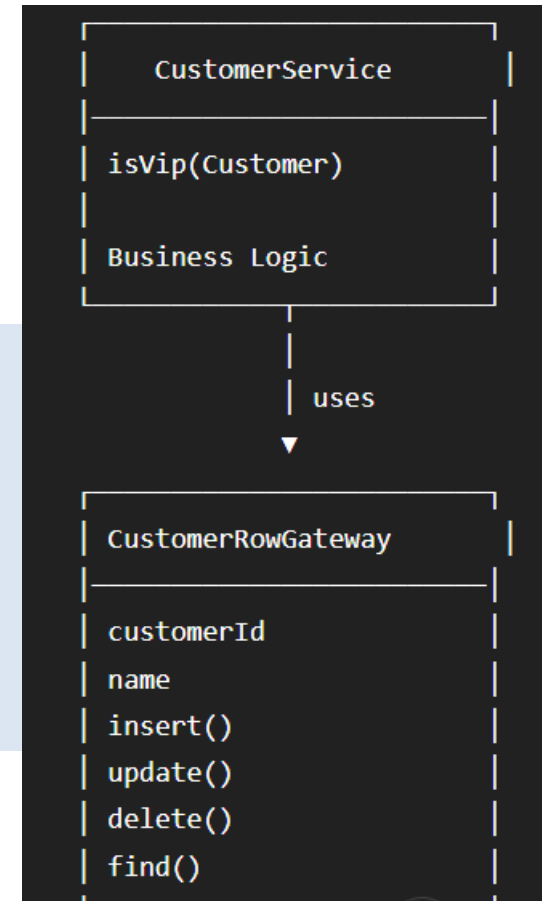
//business logic ควรต้องอยู่อีก **layer** หรือ **class** อื่น

```
class CustomerService {
```

```
    public boolean isVip(CustomerRowGateway customer) {  
        return ...;
```

```
    }
```

```
}
```



Active Record

An object that wraps a row in a database table or view, encapsulates the database access, and **adds domain logic on** that data.

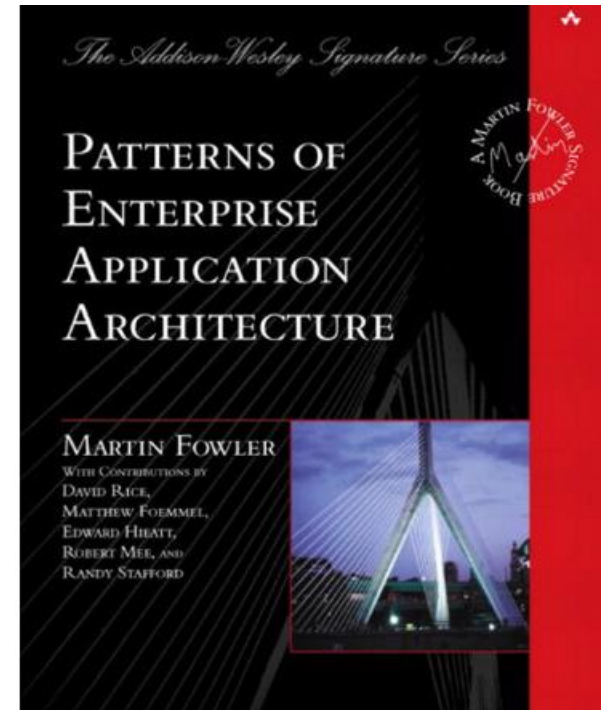
Customer object เองรู้วิธีเข้าถึง database

Customer Active Record

```
|— customerId
|— name
|— insert()
|— update()
|— delete()
|— find()
|— isVip()      ← Business Logic
```

Order Active Record

```
|— orderId
|— customerId
|— amount
|— insert()
|— update()
|— delete()
|— find()
```



```
public class Customer {
```

```
    private long customerId;
```

```
    private String name;
```

แบบที่ 1

```
    public boolean isVip(CustomerRowGateway customer) {
```

```
        return ...;
```

```
    }
```

```
    public void insert(Connection connection) {
```

```
        String sql = "INSERT INTO CUSTOMER (customer_id, name) VALUES (?, ?)";
```

```
        PreparedStatement ps =
```

```
            connection.prepareStatement(sql) {
```

```
                ps.setLong(1, customerId);
```

```
                ps.setString(2, name);
```

```
                ps.executeUpdate();
```

```
    }
```

```
    public static Customer find( Connection connection, long customerId) {
```

```
        String sql = "SELECT customer_id, name FROM CUSTOMER WHERE customer_id = ?";
```

```
        PreparedStatement ps = connection.prepareStatement(sql)
```

```
            ps.setLong(1, customerId);
```

```
            ResultSet rs = ps.executeQuery() {
```

```
                if (rs.next()) {
```

```
                    return new Customer( rs.getLong("customer_id"), rs.getString("name") );
```

```
                }
```

```
    }
```

```
public class Order {
```

```
    private long orderId;
```

```
    private long customerId;
```

```
    private double amount;
```

```
    public void insert(Connection connection) {
```

```
        String sql = "INSERT INTO ORDERS (order_id, customer_id, amount) VALUES (?, ?, ?)";
```

```
        PreparedStatement ps = connection.prepareStatement(sql) {
```

```
            ps.setLong(1, orderId); ps.setLong(2, customerId); ps.setDouble(3, amount);
```

```
            ps.executeUpdate();
```

```
        }
```

```
    }
```

```
    public static Order find( Connection connection, long orderId) {
```

```
        String sql = "SELECT order_id, customer_id, amount FROM ORDERS WHERE order_id = ?";
```

```
        PreparedStatement ps = connection.prepareStatement(sql)
```

```
            ps.setLong(1, orderId);
```

```
        ResultSet rs = ps.executeQuery()
```

```
            if (rs.next()) {
```

```
                return new Order( rs.getLong("order_id"), rs.getLong("customer_id"),
```

```
                    rs.getDouble("amount")
```

```
                );
```

```
            return null;
```

```
    }
```

```
Connection connection = DB.getConnection();
```

```
Customer customer = Customer.find(connection, 101);  
customer.setName("Somchai Jaidee");  
customer.update(connection);
```

```
Order order = new Order( 1001, 101, 500.00 );  
order.insert(connection);
```

```
List<Order> orders = Order.findByCustomerId( connection, 101 );
```

```
connection.close();
```

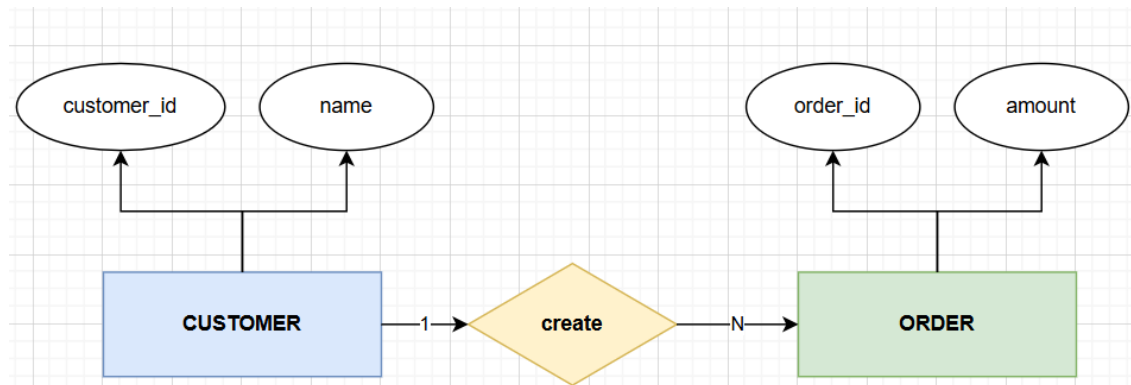
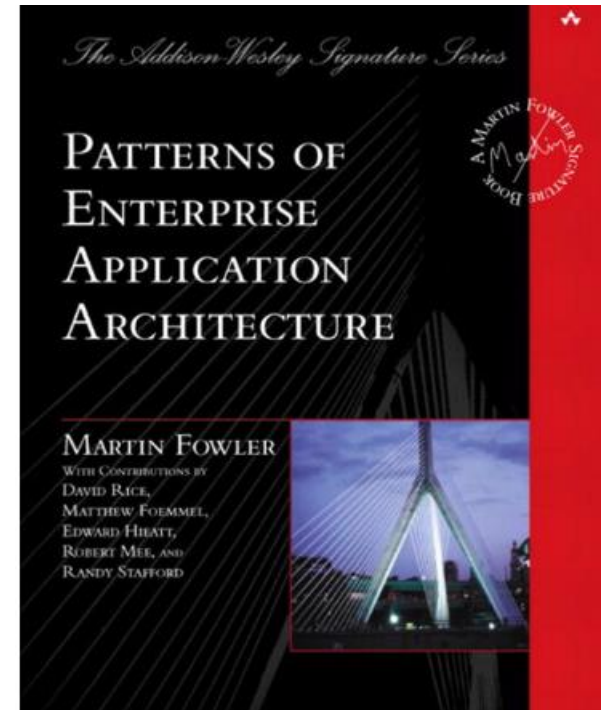
```
class Customer {  
  
    private long customerId;  
    private String name;  
    private List<Order> orders;  
  
    public boolean isVip(CustomerRowGateway customer) {  
        return ...;  
    }  
  
    public void insert(Connection connection) throws SQLException {  
        // INSERT CUSTOMER  
    }  
  
    public void update(Connection connection) throws SQLException {  
        // UPDATE CUSTOMER  
    }  
  
    public void delete(Connection connection) throws SQLException {  
        // DELETE CUSTOMER  
    }  
}
```

```
class Order {  
    private long orderId;  
    private long customerId;  
    private double amount;  
  
    public void insert(Connection connection) throws SQLException {  
        // INSERT ORDER  
    }  
  
    public static Order find( Connection connection, long orderId) throws SQLException {  
        // SELECT ORDER  
    }  
  
    public static List<Order> findByCustomerId( Connection connection, long customerId)  
  
        // SELECT ORDERS  
        // WHERE customer_id = ?  
    }  
    public void update(Connection connection) throws SQLException {  
        // UPDATE ORDER  
    }  
    public void delete(Connection connection) throws SQLException {  
        // DELETE ORDER  
    }  
}
```

Data Mapper

layer of mappers that moves data between objects and a database while keeping them independent of each other and the mapper itself.

Customer ห้ามมี SQL



```
class Customer {  
    private long customerId;  
    private String name;  
    private List<Order> orders;
```

```
    public boolean isVip( ) {  
        return ...;  
    }  
}
```

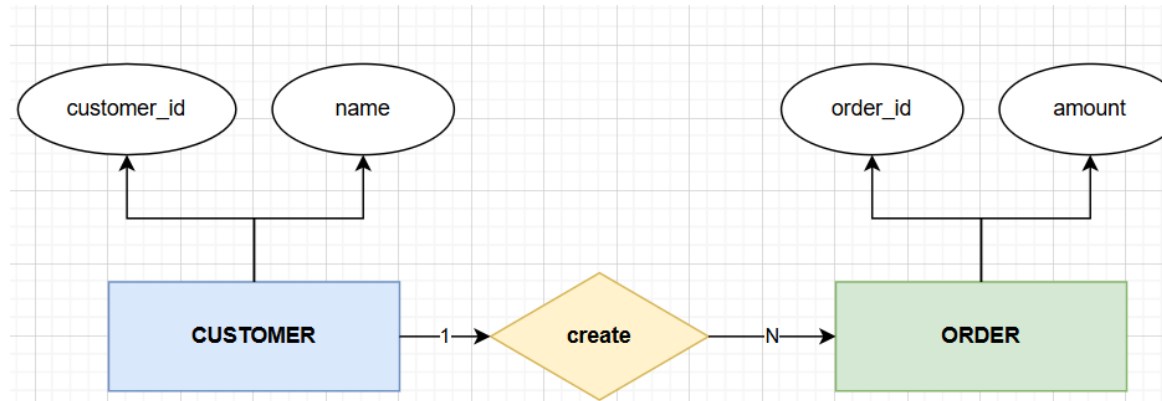
```
    public List<Order> getOrders() {  
        return orders;  
    }
```

```
    public void setOrders(List<Order> orders) {  
        this.orders = orders;  
    }
```

```
    public long getCustomerId() {  
        return customerId;  
    }
```

```
    public void setCustomerId(long customerId) {  
        this.customerId = customerId;  
    }
```

```
    public String getName() {  
        return name;  
    }
```



Method #1

```
class Order {  
    private long orderId;  
    private double amount;  
    private Customer customer;  
  
    public long getOrderId() {  
        return orderId;  
    }  
    public void setOrderId(long orderId) {  
        this.orderId = orderId;  
    }  
    public double getAmount() {  
        return amount;  
    }  
    public void setAmount(double amount) {  
        this.amount = amount;  
    }  
    public Customer getCustomer() {  
        return customer;  
    }  
    public void setCustomer(Customer customer) {  
        this.customer = customer;  
    }  
}
```

```
class CustomerMapper {
```

```
    public Customer findById( Connection connection, long customerId) {
```

```
        String sql = "SELECT customer_id, name FROM customer WHERE customer_id = ?";
```

```
        PreparedStatement ps = connection.prepareStatement(sql);
```

```
        ps.setLong(1, customerId);
```

```
        ResultSet rs = ps.executeQuery();
```

```
        if (!rs.next()) {
```

```
            return null;
```

```
        }
```

```
        Customer customer = new Customer();
```

```
        customer.setCustomerId( rs.getLong("customer_id"));
```

```
        customer.setName( rs.getString("name"));
```

```
        return customer;
```

```
    }
```

```
}
```

```
class OrderMapper {  
  
    public List<Order> findByCustomer(Connection connection, long customerId) {  
  
        String sql = "SELECT order_id, amount FROM orders WHERE customer_id = ?";  
  
        PreparedStatement ps = connection.prepareStatement(sql);  
        ps.setLong(1, customerId);  
        ResultSet rs = ps.executeQuery();  
  
        List<Order> orders = new ArrayList<>();  
        while (rs.next()) {  
  
            Order order = new Order();  
            order.setOrderId( rs.getLong("order_id"));  
            order.setAmount( rs.getDouble("amount"));  
            orders.add(order);  
        }  
        return orders;  
    }  
}
```

CustomerMapper customerMapper = new CustomerMapper();

OrderMapper orderMapper = new OrderMapper();

Customer customer = customerMapper.findById(connection, 1);

**List<Order> orders = orderMapper.findByCustomer(connection,
customer.getCustomerId());**

customer.setOrders(orders);

```
class Customer {
```

```
    private long customerId;
```

```
    private String name;
```

```
    private List<Order> orders;
```

```
    public List<Order> getOrders() {  
        return orders;  
    }
```

```
    public void setOrders(List<Order> orders) {  
        this.orders = orders;  
    }
```

```
    public long getCustomerId() {  
        return customerId;  
    }
```

```
    public void setCustomerId(long customerId) {  
        this.customerId = customerId;  
    }
```

```
    public String getName() {  
        return name;  
    }
```

```
    public void setName(String name) {  
        this.name = name;  
    }
```

```
}
```

//business logic ควรต้องอยู่อีก layer หรือ class อื่น

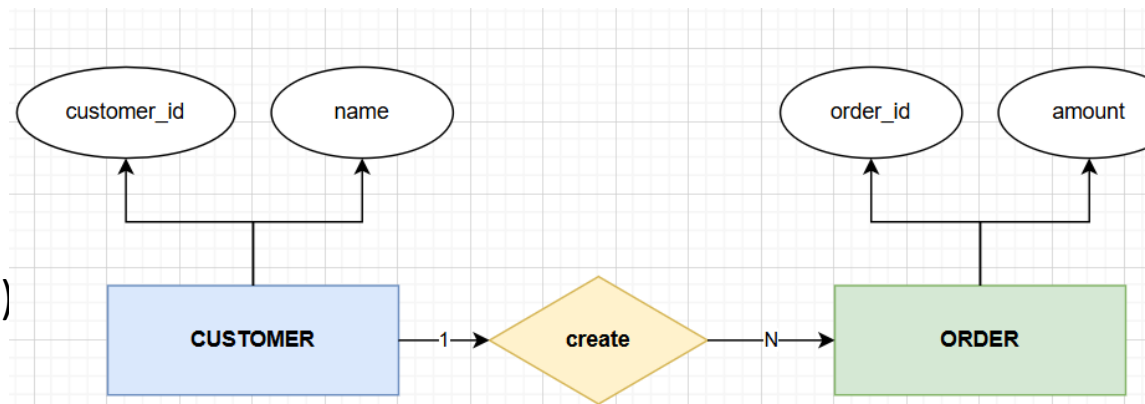
```
class CustomerService {
```

```
    public boolean isVip(Customer customer) {  
        return ...;  
    }
```

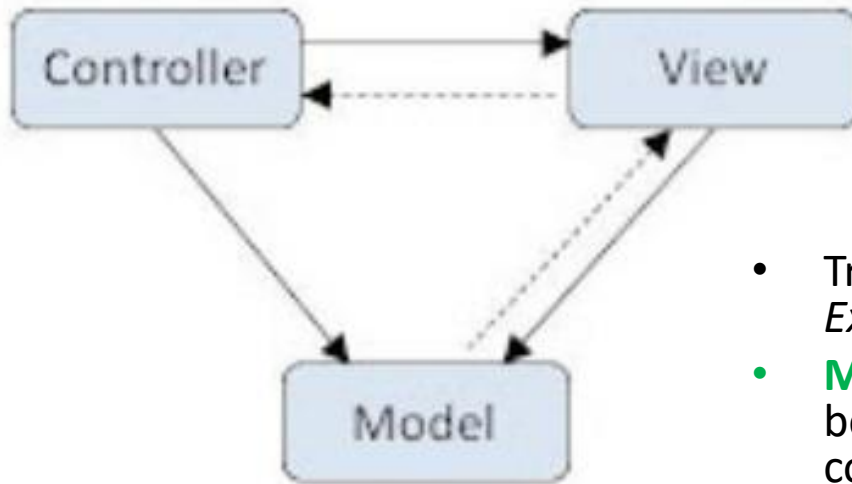
```
}
```

Data Mapper ต้องใช้ CustomerService สำหรับ business logic

Method#2

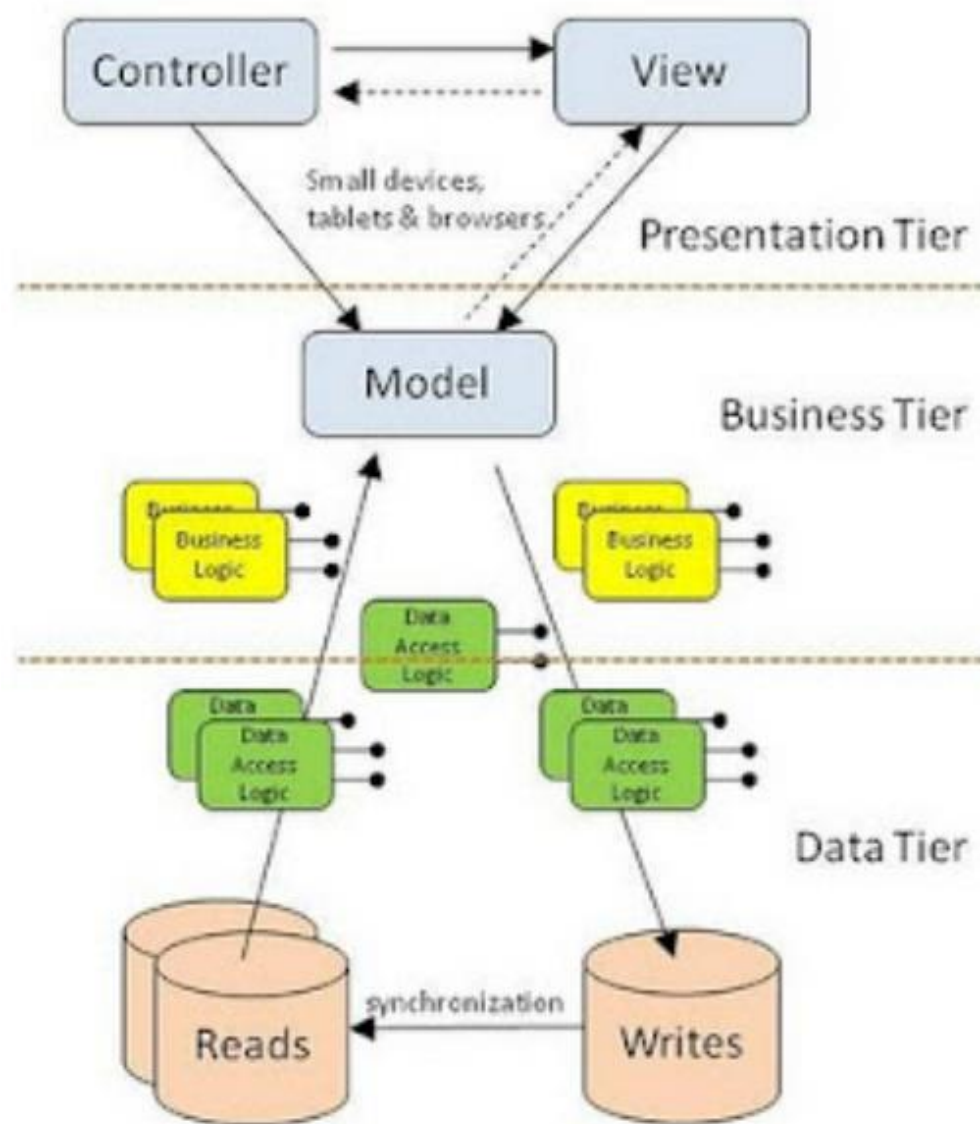


MVC



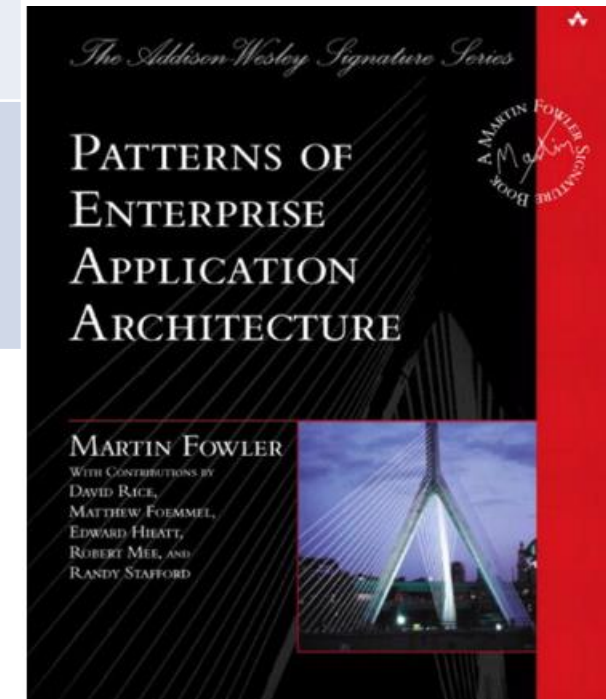
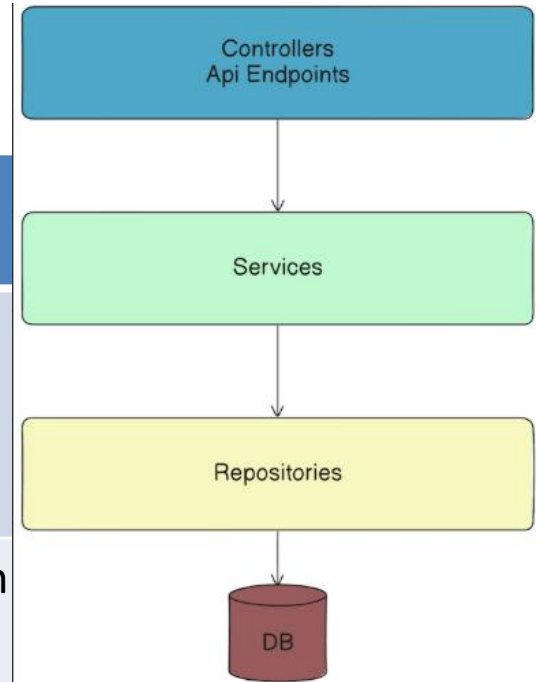
The MVC Pattern

- Trygve Reenskau, *Thing-Model-View-Editor – an Example from a planning system*, **12 May 1979**.
- **M**: Models represent knowledge. A model could be a single object (rather uninteresting), or it could be some structure of objects
- **V**: A view is a (visual) representation of its model. It would ordinarily highlight certain attributes of the model and suppress others. It is thus acting as a presentation filter.
- **C**: A controller is the link between a user and the system. It provides the user with input by arranging for relevant views to present themselves in appropriate places on the screen. It provides means for user output by presenting the user with menus or other means of giving commands and data



The MVC - Three Tier Hybrid

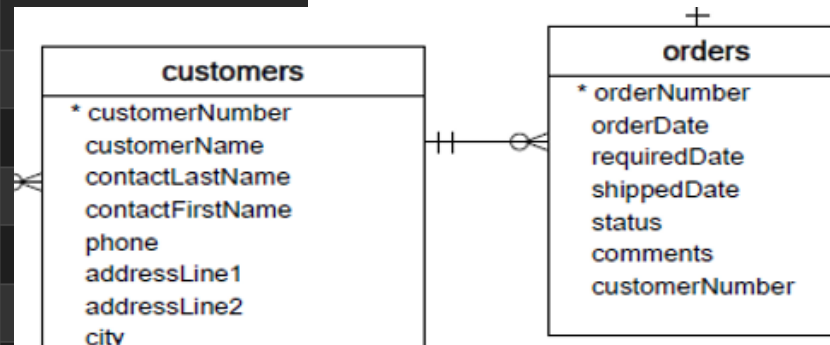
Layered Architecture



Pattern	Fowler จัดอยู่ในหมวด	Definition ของ Fowler
Service Layer	Domain Logic Patterns	“Defines an application's boundary with a layer of services...”
Repository	Object-Relational Structural Patterns	“Mediates between the domain and data mapping layers...”
Data Transfer Object	Distribution Patterns	“An object that carries data between processes...”

	customernumber [PK] integer	customername character varying (50)	contactlastname character varying (50)	contactfirstname character varying (50)	phone character varying (50)	addressline1 character varying (50)	addressline2 character varying (50)
1	103	Atelier graphique	Schmitt	Carine	40.32.2555	54, rue Royale	[null]
2	112	Signal Gift Stores	King	Jean	7025551838	8489 Strong St.	[null]
3	114	Australian Collectors, Co.	Ferguson	Peter	03 9520 4555	636 St Kilda Road	Level 3
4	119	La Rochelle Gifts	Labruno	Janine	40.67.8555	67, rue des Cinquante Otages	[null]
5	121	Deane Mini Imports	Desaulniers	Jane	03 99 0555	Eding Skeldres, apt 70	[null]

	customernumber integer	ordernumber [PK] integer	orderdate timestamp without time zone	status
1	103	10123	2003-05-20 00:00:00	
2	103	10345	2004-11-25 00:00:00	
3	103	10298	2004-09-27 00:00:00	
4	112	10346	2004-11-29 00:00:00	
5	112	10124	2003-05-21 00:00:00	
6	112	10278	2004-08-06 00:00:00	Shipped
7	114	10342	2004-11-24 00:00:00	Shipped
8	114	10347	2004-11-29 00:00:00	Shipped
9	114	10125	2003-05-21 00:00:00	Shipped



Menu

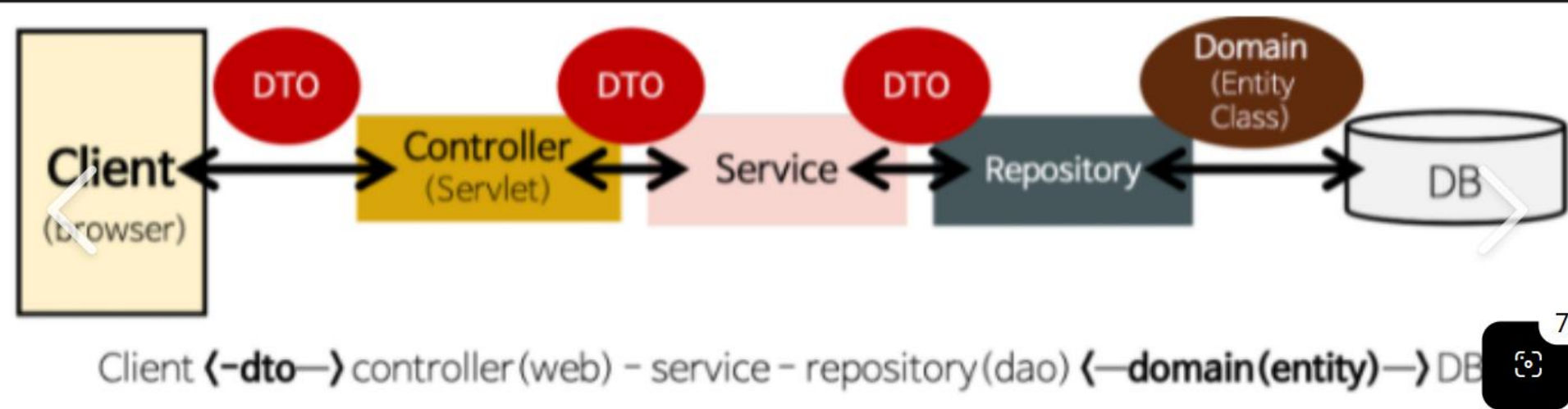
view Jdbc order-customer-count list

Show 10 entries

Search:

customerNumber	customerName	CountOrder
103	Atelier graphique	3
112	Signal Gift Stores	3
114	Australian Collectors, Co.	5
119	La Rochelle Gifts	4
121	Baane Mini Imports	4
124	Mini Gifts Distributors Ltd.	17
128	Blauer See Auto, Co.	4
129	Mini Wheels Co.	3
131	Land of Toys Inc.	4
141	Euro+ Shopping Channel	26
customerNumber	customerName	CountOrder

Architecture



https://github.com/rapirakgmail/COS3103_169/tree/main

- Entity Framework (JpaCustomerOrderRepository)
- View Create View approach (VCustomerOrderRepository)
- Repository Query (JdbcCustomerOrderRepository)